

Exploring Hybrid Memory B+-Trees: Profiling, Design Decisions, and Trade-offs

Arjun Babu Chandra Mouleshwari Surendra Babu

*Faculty of Computer Science
Otto von Guericke University
Magdeburg, Germany
arjun.chandra@st.ovgu.de*

Atif Harshad

*Faculty of Computer Science
Otto von Guericke University
Magdeburg, Germany
atif.harshad@st.ovgu.de*

Ijaaz Muhammed Mullamangalam

*Faculty of Computer Science
Otto von Guericke University
Magdeburg, Germany
ijaaz.mullamangalam@st.ovgu.de*

Karthik Varma Nandakumar Rama Varma

*Faculty of Computer Science
Otto von Guericke University
Magdeburg, Germany
karthik.nandakumar@st.ovgu.de*

Nithin Baiju

*Faculty of Computer Science
Otto von Guericke University
Magdeburg, Germany
nithin.baiju@st.ovgu.de*

Vanessa Tojan

*Faculty of Computer Science
Otto von Guericke University
Magdeburg, Germany
vanessa.tojan@st.ovgu.de*

Abstract—With the introduction of Non-Volatile Memory (NVM) as a high performance alternative to traditional DRAM and NAND flash storage, optimizing data structures for hybrid memory environments has become a critical research area. In this paper, an NVM-optimized B+-tree is compared with the FP-tree, which is a hybrid B+-tree specifically designed for NVM-DRAM systems. Through in-depth micro-benchmarking and profiling, we analyze key performance indicators like operation latencies, CPU consumption, and memory efficiency to understand the trade-offs between these data structures. Our findings indicate that FP-tree outperforms NVM-optimized B+-tree in insertion, search, and delete latencies across the board due to its selective write-back policy, light-weight logging techniques, and hybrid memory optimization. Furthermore, Intel VTune profiling and other profiling tools reveal critical memory allocation bottlenecks and CPU inefficiency for both trees. To further enhance the efficiency of the B+-tree, we recommend the addition of zero-copy deserialization using the Cista library, with the goal of reducing serialization overhead and providing improved performance. Future work will also include supporting variable-length string datasets more broadly, enabling more use across real-world workloads. This paper provides informative data on how to improve persistent tree structures for hybrid memory systems and offers a foundation for further development in database storage management and indexing.

Index Terms—NVM, DRAM, B+-tree, Zero-copy deserialization, Benchmarking, Profiling

I. INTRODUCTION

Traditional storage architectures which utilises DRAM and NAND flash has limitations. DRAM is fast, but it is volatile and does not scale well for tasks which are data-intensive. NAND flash is persistent but is limited by slow speeds and limited endurance. These inefficiencies inspire the development of hybrid memory solutions, like those which integrates non-volatile memory (NVM), to eradicate the performance gap between DRAM and persistent storage. As laid out in Emerging NVM: A Survey on Architectural Integration and Research Challenges, this architectural difference motivates the development of hybrid memory solutions including

newer technologies such as non-volatile memory (NVM). [Chen(2016)]

NVM remodels memory technology by inculcating the speed of DRAM with the persistence that can be provided by NAND flash. This non-volatile memory saves data without power, providing solutions to issues like loss of data during crashes and minimising the overhead of maintaining consistency in storage systems. Persistent data structures such as LB-Trees and NV-Trees [Lee et al.(2014)] aid these capabilities to reach maximum consistency and reduce write amplification. But these structures face challenges in memory utilization, cache efficiency, and durability, particularly in hybrid environments combining DRAM and NVM. [Oukid et al.(2016)]

The B+-tree is a self-balancing search tree which is important for efficient data retrieval in databases [Oukid et al.(2016)]. It performs very well in point and range queries because of its organization of keys in internal nodes and data in linked leaf nodes. Its scalability originates from high space utilization, minimizing splits and merges during updates. Although adapting B+-trees to NVM requires overcoming challenges like write amplification and cache inefficiencies, it can be solved with hybrid memory-aware optimizations [BOUKHOBZA et al.(2017)]. Through an efficient caching mechanism and page replacement policy, hybrid B+-trees optimized for NVM mitigate these challenges by minimizing write amplification and improving performance. These optimizations form the foundation for further exploration of B+-tree variants fine tuned for hybrid memory systems. Our research uses an NVM-optimized B+-tree as a fundamental data structure for upcoming tests, including system usage profiling, micro-benchmarking tree operations..

The FP-tree is a hybrid B+-tree optimised for NVM-DRAM systems, using DRAM as a cache and SCM for persistent storage [Oukid et al.(2016)]. By minimising SCM writes with a selective write-back policy and employing lightweight

logging, the FP-tree reduces write amplification, enhances durability, and supports concurrency and crash consistency. This makes it a highly efficient data structure that addresses limitations of traditional persistent structures and this state-of-the-art FP-tree implementation is the second data structure in consideration for comparison studies with our recently developed NVM optimized B+-tree.

Profiling is the process of checking an application to gather performance-related data, which in turn helps in identifying execution inefficiencies and bottlenecks. According to the type of analysis that needs to be done, different tools are available for profiling. [Kakaraparthi and Patel(2024)] discusses hardware-level profiling tools, which provide a detailed view into system performance but may cause significant overhead and require specialised knowledge. [Debbarma and Chandrasekaran(2016)] includes perf, a commonly used open-source tool for CPU profiling in Linux environments. perf is useful for profiling system-level performance, providing a range of metrics like cache hits/misses and instruction cycles, but it does not offer detailed information regarding memory or GPU-specific profiling like other tools. VTune Profiler, as [Tsymbal and Kurylev(2017)], provides profiling for both CPU and GPU, offering low overhead and detailed performance metrics, especially for multi-threaded and parallel applications. Perf is lightweight and easy to use, while VTune excels in more complex and fine-grained performance analysis. Our project mainly utilises Intel VTune Profiler for memory and CPU profiling, supported by perf and Valgrind wherever necessary to understand the memory characteristics of our data structures.

The structure of the paper is as follows, Section II gives a brief introduction to the Background of B+-trees, FP-Trees, Tools for profiling, and so on followed by Section III, which elaborates on the methodology used to implement and make this project a success. Further, in Section IV, we present the detailed highlights of our profiling and benchmarking processes to generate performance results. Later, in Section V, we present the evaluation results. Finally, in Section VI, we present a review of existing literature. Followed by the final sections, which include Conclusion, Future Work, Acknowledgments, and References of the literature.

II. LITERATURE REVIEW

In this section, we will dive deeper into the details of NVM-DRAM hybrid memory, how to conduct profiling using different tools, how to benchmark data structures, and the basics of memory technologies, tree structures, and profiling parameters.

A. 2.1 DRAM (Dynamic Random Access Memory)

DRAM's fast performance and scalability make it the building block of contemporary computing. But because DRAM technology has other physical limitations like higher power consumption, problems with data retention, and refresh overhead have grown to be serious issues. DRAM cell convergence results in smaller capacitors, which negatively impacts data

retention and necessitates more frequent refresh cycles, which uses more power. Furthermore, due to its instability DRAM is not suitable for permanent storage applications, which causes data loss when power is switched off.

[Spessot and Oh(2020)] Discusses these limitations, emerging memory technologies like Storage-Class Memory (SCM) and Non-Volatile Memory (NVM) offer promising alternatives. SCM bridges the gap between DRAM and traditional storage by providing near-DRAM speed with persistent storage capabilities. Similarly, NVM technologies, such as Phase-Change Memory (PCM) and Resistive RAM (ReRAM), offer non-volatility while maintaining relatively fast access times, reducing dependency on continuous refresh operations. By integrating SCM and NVM into memory hierarchies, systems can reduce power consumption, improve data retention, and enhance overall efficiency, making them viable solutions for the challenges facing DRAM today.

B. 2.2 NVM (Non-Volatile Memory)

NVM is a new kind of memory device that offers high-capacity, long-lasting storage that is byte addressable. NVM, which is located on the memory bus, enables the building of robust data structures that are comparable to their in-memory counterparts. Expensive serialization/deserialization operations, usually associated with block-based storage, are not necessary. Unfortunately, using NVM is not as simple as placing a data structure in NVM and expecting persistence. As NVM sits on the memory bus, processor caches buffer the cache lines from it that are referenced by load and store instructions.. While NVM offers slower speeds compared to DRAM, it excels in data persistence and durability, making it crucial for storage and archival purposes.

A new kind of memory that provides quick, byte-addressable, long-lasting storage is called non-volatile memory (NVM). With load and store instructions, pointer-based data structures like hash maps and B+ trees can be directly stored and manipulated in Nhttps://www.overleaf.com/latex/templates/-/wbvngjhbwpcVM. Even in the event of a process crash or power outage, these structures remain accessible. [Zaki and Hsiao([n.d.])]

C. 2.3 SCM (Storage-Class Memory)

SCM is a kind of non-volatile memory that bridges the gap between NAND flash storage and dynamic random-access memory (DRAM). It is appropriate for applications requiring high-performance memory with permanent store capabilities because it strikes a balance between speed, endurance, and cost. SCM, as opposed to DRAM, offers faster access times than NAND flash while preserving data permanence, in contrast to conventional storage options. This makes it a great option for increasing overall computing architecture efficiency, decreasing system latency, and speeding up data-intensive tasks.

Storage-type SCM and memory-type SCM are the two primary categories into which SCM falls. Storage-type SCM

functions as a quicker and more robust substitute for NAND flash since it is cost and endurance optimised. In contrast, memory-type SCM prioritises speed and functions similarly to DRAM while preserving non-volatility. SCM is now more likely to be widely used in contemporary computer systems thanks to developments in materials, thermal control, and scaling strategies. SCM is anticipated to be essential in closing the performance gap between memory and storage systems as the need for quicker and more effective data storage increases.

D. 2.4 B+- Tree

The B+-tree is a complex balanced tree-based data structure that is particularly designed to facilitate the efficient storage and retrieval of large amounts of data. Its design is particularly well-suited for disk storage or non-volatile memory (NVM) systems where performance is significantly based on data access patterns. Important studies, such as those by [Comer(1979)] provide a clear insight into the architecture and operation of the B+ tree.

This data structure is made up of three principal node types: internal nodes, leaf nodes, and, in some implementations, a root node. Internal nodes are the hub of the navigation process; they contain keys that serve as separators and pointers that reference child nodes. By keeping these keys sorted, internal nodes efficiently direct search operations along the tree.

Leaf nodes, or the end nodes of the tree, are linked together in a linked list that gives quick, sequential access to the information that they store. The nodes store the data itself as key-value pairs (or keys pointing to data pointers), so retrieving individual values is simple and cost-effective.

Each node in the B+-tree is built to carry a maximum of N keys, the order of the tree. This is important because this keeps the structure balanced; whenever nodes receive more than they can carry, they split to create new nodes, and occasionally nodes merge when they get thinly populated. This dynamic balancing maintains the tree height lower, giving logarithmic time complexity— $O(\log N)$ —for search, insertion, and deletion operations. [Krótkiewicz and Mikłasz(2020)]

One of the most important aspects of the B+-tree is that all leaf nodes are at the same level, making it possible to access any key with uniform time efficiency. This uniformity in the design allows for quick data retrieval for users independent of the data's position in the dataset. Overall, the B+ tree's design philosophy is to allow for maximum performance and scalability, thus making it a very suitable data handling structure for applications managing enormous amounts of data.

E. 2.5 Fingerprinting Persistent Tree (FPTree)

The structure of persistent data structures creates a number of challenges for maintaining the data consistent, especially considering that Storage Class Memory (SCM) is accessed through the volatile CPU cache, which provides limited control to the program. The Fingerprinting Persistent Tree (FPTree) tries to address this by employing four main design principles in an effort to achieve performance close to DRAM-based data structures [Oukid et al.(2016)]:

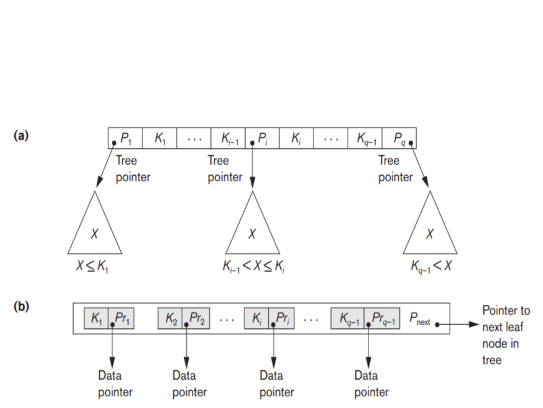


Fig. 1: B+-Tree Structure [R and B.(2016)]

1) *Fingerprinting*: This plan involves the construction of one-byte hashes, called fingerprints, of the leaf-node keys. Fingerprints are packed sequentially into the first cache-line-sized portion of the leaf.

2) *Selective Persistence*: This approach is to store primary data in SCM and non-primary data in DRAM. In the case of the FPTree, it means storing the leaf nodes in SCM and inner nodes in DRAM. So, in tree traversal, the leaf node accesses only incur higher costs than with a fully transient data structure.

3) *Selective Concurrency*: This concept is based on employing different concurrency models for the transient and persistent aspects. Specifically, the FPTree utilizes Hardware Transactional Memory (HTM) to manage internal node concurrency, and fine-grained locks for leaf nodes.

4) *Amortized Persistent Memory Allocations*: Amortized Persistent Memory Allocations - Allocations of persistent memory can be costly, particularly on splits when new leaves have to be allocated. To combat this, we implement an amortization technique through the allocation of batches of several leaves simultaneously. This is achieved through two data structures: a linked list of batches of currently allocated leaves and a volatile array of leaves which are presently free and unallocated in the tree. [Oukid et al.(2016)]

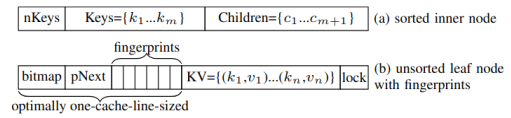


Fig. 2: FPTree inner node and leaf node layouts [Oukid et al.(2016)]

SCMs are also burdened with several challenges that hinder their wider adoption. The main drawback is the low write endurance of SCM devices, which leads them to wear out prematurely with repeated updates. Although the latency of SCMs is better than traditional storage, it is still greater than DRAM, thereby leading to performance bottlenecks for certain workloads. Additionally, the complexity of handling data is added to by SCMs' need for sophisticated algorithms for efficient memory management, garbage collection, and wear leveling. Such limitations call for innovative solutions that

ensure optimal SCM use while maintaining performance and lifespan.

The FP-Tree (Frequency Pattern Tree) data structure offers a viable solution to solving many of these SCM-related issues. By utilizing its hierarchical organization, the FP-Tree decreases the number of writes employed for updating or reading data and hence reduces wear on SCM devices. This is achieved through compressed representation and compression techniques that support more efficient storage of frequent patterns, thus leading to minimized memory footprint and access times. Second, the FP-Tree is structured to have inherent parallel support, which makes traversal and handling of large amounts of data at faster speeds possible, consistent with SCMs benefits. Consequently, the incorporation of FP-Tree structures in SCM-based systems proves to be an appropriate approach for bridging existing technology boundaries and realizing next-generation memory technology's full potential. [Oukid et al.(2016)]

F. 2.6 Profiling and Tools

Profiling is a performance analysis tool with a critical role in software development that is mostly concerned with the analysis of a program's execution to measure the most important performance metrics. The measurements such as latency, throughput, and memory usage enable us to identify performance bottlenecks and provide insight on how to optimize. With the understanding of how a program uses resources like CPU cycles, memory, and I/O, we can make certain adjustments to optimize the overall efficiency and the performance, especially in data-intensive programs.

1) **2.6.1 Perf:** Perf is a comprehensive and heavy-duty performance analyzing tool built within the Linux OS. It's extremely well-known for providing system-wide and application-specific low-level performance data. Perf, as part of Linux's performance set of tools, is used far and wide in identifying software bottlenecks using sampling of variety of hardware as well as software events. Those events are the CPU cycles, cache misses, branch mispredicts, page faults, as well as the kernel events of context switches as well as system calls.

2) **2.6.2 Intel VTune Profiler:** Intel VTune Profiler is a commercial performance analysis product developed by Intel with robust measurement and optimization capabilities for Intel processors, but whose capabilities are lessened on other platforms. One of its strongest capabilities, hotspot analysis, helps make it easier to identify code hotspots with long run times by creating a list of functions in order of run time. VTune is also capable of tracing the call stack of all functions, providing information on how performance-critical functions are called.

In addition, when profiling parallel programs, VTune enables the identification of communication hotspots in MPI routines and helps uncover communication patterns and inefficiencies. Based on these observations, developers can optimize communication strategies, reduce overhead, and maximize computation-communication overlap. Identification of

an application's hotspot routines and call chains allows the identification of performance bottlenecks and optimization opportunities, ultimately leading to better system performance [Yang et al.(2024)]

III. METHODOLOGY AND IMPLEMENTATION

This section outlines the approaches taken to conduct a comparative analysis of an NVM-optimized B+-tree with an FP-tree(NVM-DRAM hybrid memory optimized). This is done by running a set of micro-benchmarks on both the B+-tree and FP-tree in order to obtain latencies for these tree operations - insert, search and delete. Then the NVM-optimized B+-tree is profiled using a suitable profiling tool to obtain results with focus on Memory Usage and CPU Utilization. From these tests on the B+-tree and FP-tree we aim to obtain the performance behavior of both the trees and perform an in-depth comparison study. In this section we will provide a brief description of each task in detail.

A. 3.1 FP-tree Micro-benchmarks

The FP-tree, a hybrid SCM-DRAM persistent and concurrent B-Tree proposed in [Oukid et al.(2016)] is the chosen tree for comparison studies. Micro-benchmark reproducibility of the tree are performed as published by the authors. In our case, we only utilize the single-threaded FP-tree for micro-benchmarking. The scope of application of our project is restricted to fixed-size keys, i.e., both keys and values are 8-byte integers. We execute the single-threaded binary for the tree to get latencies per operation with fixed-size keys as input.

1) **3.1.1 Input Data :** Input data files are produced by running an authors-provided script. Two data files(one for integers, and others for strings) contain uniformly distributed integers/16-byte strings, respectively. Since we are concerned with keys of a fixed size, we need only the integer data file. Through the running of a data generator script, we produced uniformly distributed 100000000 keys, to be input to our FP-tree.

2) **3.1.2 Parameters:** In this sub section, the parameters considered for the FP-tree are discussed.

a) **3.1.2.1 Number of Keys:** This denotes the amount of keys to be transmitted to the tree per operation to be performed. We will progress gradually warming the tree with 50000, 500000, 5000000 and 50000000 keys correspondingly in a bid to capture latencies per operation.

Parameters	Ranges
Number of Keys	50000 - 50000000

TABLE I: Parameter values for FP-tree

B. 3.2 B+-tree Micro-benchmarks

Performance analysis of our B+-tree optimized for NVM is the primary concern of this project. The tree in question is copied from a C++ project which is an ongoing endeavor to investigate and implement various Bε-Tree and B+-Tree

variations, accessible on GitHub ¹. In our case, we consider only the single-threaded variant of the B+-tree for micro-benchmarking. Further, the domain of our project is restricted to fixed-size keys, i.e., keys and values are 8-byte integers too. We execute the single-threaded variant of the binary of the tree to obtain latencies for each operation, with fixed-size keys as input. A generic shell script is employed that executes the binary of the tree for each parameter configuration provided below. For each execution of the binary called by the script, we obtain the total execution time and the latency of each operation in a separate log file.

1) **3.2.1 Input Data** : The same data file, i.e., a uniformly distributed 100000000 keys used for micro-benchmarking FP-tree is used as the input to our B+-tree.

2) **3.2.2 Parameters**: In this sub section, the parameters considered for the B+-tree are discussed.

a) **3.2.2.1 Number of Keys**: The number of keys being passed to the tree for each operation to be performed. We will warm up the tree with 50000, 500000, 5000000 and 50000000 keys successively to retrieve latencies for each operation.

b) **3.2.2.2 Tree Degree**: Degree in a B+-tree refers to the maximum number of children that a node can have, which in turn determines the tree's branching factor. The greater the degree, the lower the number of levels and therefore the faster the lookups, insertions, and deletions with more keys stored per node. Our script executes the binary for degrees from 32 up to 1024 by steps of 32 each time.

Parameters	Ranges
Number of Keys	50000 - 50000000
Degree	32 - 1024

TABLE II: Parameter values for B+-tree

c) **3.2.2.3 Buffer Size**: The number that denotes size of volatile buffer pool or cache DRAM in memory, that affects how much data remains in memory vs on disk. It is not the size of a storage but its impact is in performance. Its dynamic computation comes based on keys' quantity and degree. This number is presented in terms of Megabytes(MB).

1. Internal Node Size:

$$\begin{aligned} \text{nInternalNodeSize} = & (\text{nDegree} - 1) \times \text{sizeof}(\text{ValueType}) \\ & + \text{nDegree} \times \text{sizeof}(\text{ObjectUIDType}) \\ & + \text{sizeof}(\text{int}^*) \end{aligned} \quad (1)$$

2. Total Internal Nodes:

$$\text{nTotalInternalNodes} = \frac{\text{nMaxNumber}}{\text{nDegree}} \quad (2)$$

3. Total Memory:

$$\text{nTotalMemory} = \text{nTotalInternalNodes} \times \text{nInternalNodeSize} \quad (3)$$

4. Total Memory in MB:

$$\text{nTotalMemoryInMB} = \frac{\text{nTotalMemory}}{1024 \times 1024} \quad (4)$$

¹<https://github.com/sajadKarim/haldendb/>

5. Buffer Size:

$$\text{nBufferSize} = \begin{cases} 100 & \text{if } \text{nTotalMemoryInMB} > 100 \\ 0.25 \times \text{nTotalMemoryInMB} & \text{otherwise} \end{cases} \quad (5)$$

d) **3.2.2.4 Block Size**: Defines the size of each block or page used for node storage to avoid memory or disk misalignment. Storage must be capable of holding these sized blocks. This is also calculated dynamically. It is in units of kilobytes(KB).

$$\text{nBlocksize} = \begin{cases} 256 & \text{if } \text{nInternalNodeSize} > 256 \\ 128 & \text{otherwise} \end{cases} \quad (6)$$

e) **3.2.2.5 Total Size**: Indicates the total memory used or storage size of the tree, which can refer to the maximum storage space utilized for storing nodes, caching, or persistent storage.

$$\text{nTotalMemoryInMB} = \frac{\text{nTotalMemory}}{1024 \times 1024} \quad (7)$$

$$\text{nMaxSize} = \begin{cases} 1073741824 & \text{if } \text{nTotalMemoryInMB} < 200, \\ 5368709120 & \text{otherwise.} \end{cases} \quad (8)$$

Max storage size is decided with respect to the total memory size in MB of the tree. For example, for keys = 100000000, total tree size is around 3051 MB. So we select the maximum storage as an arbitrary value 5GB. It is also calculated dynamically based on the tree size. In GB.

Parameters	Ranges
Buffer Size	1MB - 100MB
Block Size	128KB or 256KB
Total Size	1GB or 5GB

TABLE III: Parameter values for B+-tree

C. 3.3 Profiling

In order to analyze the performance characteristics of our B+ tree implementation, we make use of **Intel VTune Profiler**, a state-of-the-art performance analyzing tool. Intel VTune profiler is chosen primarily because of its ability to display thorough information on both **CPU utilization** and **Memory Consumption**, which are the most influential factors in judging the efficiency of the implementation. The profiler's **hotspots analysis** enables us to identify computationally costly regions of the code, while the **memory-consumption analysis** enables us to track memory usage patterns, including allocations, deallocations, and peak memory usage. We can identify bottlenecks and optimize resource usage effectively thanks to these analyses.

1) **3.3.1 Hotspots Analysis:** Hotspots analysis is conducted to identify parts of the code which take most of the CPU time. Hotspots analysis is employed to find computationally intensive tasks in the B+ tree implementation such as insertion, search, and deletion. To perform hotspots analysis, one undertakes the following steps:

- 1) **Data Collection:** Using Intel VTune, we execute the application under the "hotspots" collection mode. This mode captures detailed information about CPU usage across all functions and modules. The profiling is conducted for varying configurations of the B+ tree, including different degrees (*nDegree*) and numbers of keys (*nKeys*), to ensure comprehensive coverage of performance scenarios. The results are stored in dedicated directories, organized by configuration parameters.
- 2) **Timeline Data Extraction:** After the profiling run, VTune generates a **timeline report** that breaks down CPU usage into discrete time intervals (bins). Each bin represents a specific time range and contains metrics such as *BinStartTime*, *BinEndTime*, and *CPUTime : Self*. These metrics are extracted from the timeline CSV files generated by VTune.
- 3) **Calculation of CPU Utilization:** To calculate CPU utilization percentages [Friedman([n.d.])], we use the following formula:

$$\text{CPU Utilization (\%)} = \frac{\text{CPU Time}}{\text{Number of Cores} \times \text{Bin Duration}} \times 100 \quad (9)$$

where:

- Bin Duration = Bin End Time – Bin Start Time,
- Number of Cores is hardcoded based on the system configuration (e.g., 24 cores).

For example, if a bin spans 16.628 seconds and the recorded CPU time is 104.608 seconds on a 12-core system:

$$\text{CPU Utilization (\%)} = \frac{\text{CPU Time}}{\text{Number of Cores} \times \text{Bin Duration}} \times 100 \quad (10)$$

- 4) **Visualization:** The calculated CPU utilization percentages are plotted as a line graph, with the x-axis representing time intervals (bins) and the y-axis representing CPU utilization percentages. This graph highlights periods of high computational activity and identifies potential bottlenecks.

2) **3.3.2 Memory-Consumption Analysis:** Memory-consumption analysis is performed to evaluate the memory usage patterns of the B+ tree implementation. This analysis focuses on tracking memory allocations, deallocations, and peak memory usage of individual functions within our binary during execution. The steps involved in conducting memory-consumption analysis are as follows:

- 1) **Data Collection:** Using Intel VTune, we execute the application under the "memory-consumption" collection mode. This mode captures detailed information about memory usage, including active memory consumption and total physical memory available. Similar to the

hotspots analysis, profiling is conducted for various configurations of the B+ tree to capture a wide range of performance scenarios. The results are stored in dedicated directories, organized by configuration parameters.

- 2) **Calculation of Memory Utilization:** Memory Utilization percentage is calculated for individual functions within our binary under consideration. Since our tree executes 3 operations, by running memory consumption analysis with the VTune profiler, we can interpret the memory consumed for each operation. From this, we arrive at the Memory Consumption percentages for each operation. To calculate memory consumption percentages, we use the following formula:

$$\text{Memory Consumption (\%)} = \frac{\text{Memory Used}}{\text{Total Memory Allocated}} \times 100 \quad (11)$$

where:

- Total Memory Allocated is obtained for the entire binary from the profiler.

- 3) **Visualization:** Percentages of memory consumed per function are computed and plotted as a bar graph, percentage of memory consumed on the x-axis and individual functions along the y-axis. The graph highlights functions with high memory use and where the memory is likely to be wastefully utilized.

3) **3.3.3 Data Integration and Graph Plotting:** To synthesize information obtained in both analyses and present meaningful visualizations, the following steps are taken:

- 1) **Data Parsing:** The timeline CSV files generated by VTune are parsed to extract relevant metrics such as *CPUTime : Self*, *BinStartTime*, *BinEndTime*, *MemoryUsed*, and timestamps. These metrics are then processed to ensure consistency and compatibility with visualization tools.
- 2) **Graph Design:** Two separate graphs are created:
 - The CPU utilization graph uses time intervals on the x-axis and CPU utilization percentages on the y-axis.
 - The memory consumption graph uses percentages on the x-axis and functions on the y-axis.
- 3) **Interpretation:** The CPU usage graph is analyzed to find out when the system was computationally intensive, which may indicate inefficient algorithms or suboptimal code paths. The memory usage graph is analyzed to find out when the system was memory-intensive, which may indicate excessive memory allocations or potential memory leaks.

4) **3.3.4 Significance of the Methodology:** This profiling method provides a complete picture of the performance characteristics of the B+ tree implementation. By employing Intel VTune's advanced profiling capabilities, we are able to collect detailed information on CPU and memory usage. Examination and visualization of this information enable us to identify performance bottlenecks and optimize the implementation for improved efficiency. Moreover, the formal method ensures that

the profiling is scalable and repeatable, with possible future extension to other algorithms or data structures.

IV. EVALUATION AND DISCUSSIONS

In this section, we compare the performance of the B+-tree with that of the state-of-the-art FP-tree and also provide a brief overview on our experimental setup.

A. 4.1 System Setup

The test environment for the initial experiments was a high-performance Linux-based platform with the following setup: it employed an Intel(R) Xeon(R) Gold 5220R CPU running at 2.20 GHz, which possessed 96 logical processors shared between 2 sockets, each socket containing 24 cores and 2 threads per core. The platform supported both 32-bit and 64-bit operating modes, employing a little-endian byte order. As far as memory is concerned, the system had 376 GiB of RAM with 139 GiB in use, 9.7 GiB used, and 227 GiB occupied by buffers and cache. The system also had a dedicated swap space of 8 GiB, with 746 MiB utilized at the time.

As far as storage is concerned, the system had two NVMe drives (nvme0n1 and nvme1n1), each having a total capacity of 2 TB and supporting a 512B block size. It also had a 2 GB SATA SSD (sda) and some loop devices, each with a 4K granularity and a maximum discard size of 4 GB. Although persistent memory (pmem1) was present, it did not have any allocated space. The system was secure against known attacks, such as Spectre v1 and v2, Meltdown, Retbleed, and other security problems mitigations. VT-x virtualization support was enabled to facilitate smooth virtual machine running and performance benchmarking tools. FPTreeVarBenchmark

B. 4.2 Single-thread micro-benchmarks on FP-tree

In order to test the performance of the Insert, Find, and Delete operations, we first warm up the FP-tree with 50000 key-values, and then execute back-to-back 50000 Finds, Inserts, Updates, and Deletes. We do the same for keys 500000, 50000000 and 500000000 sequentially. Figure 3 shows the latencies of each operation of the FP-tree for each key value.

C. 4.3 Single-thread micro-benchmarks on B+-tree

In order to benchmark the performance of the Insert, Find, and Delete operations, we first warm up the FP-tree with 50000 key-values, then we perform successive 50000 Finds, Inserts, Updates, and Deletes. We do the same process for keys 500000, 50000000 and 500000000 successively. Figure 4 shows the latencies of each operation of the FP-tree for each key value.

To decide the optimal shape of the B+-tree, we conducted an analysis to identify the "knee point" at which the performance of the tree for all three primary operations—insert, search, and delete—is at lowest latency prior to a drop. By performing this procedure for every key size (ranging from 50,000 to 50,000,000 keys), we were able to obtain a complete picture of the effect of the tree structure parameters on its operational effectiveness under various workloads. This organized procedure ensures that the established nDegree values provide a

valid and scalable configuration of the B+-tree that achieves its optimal performance under hybrid memory.

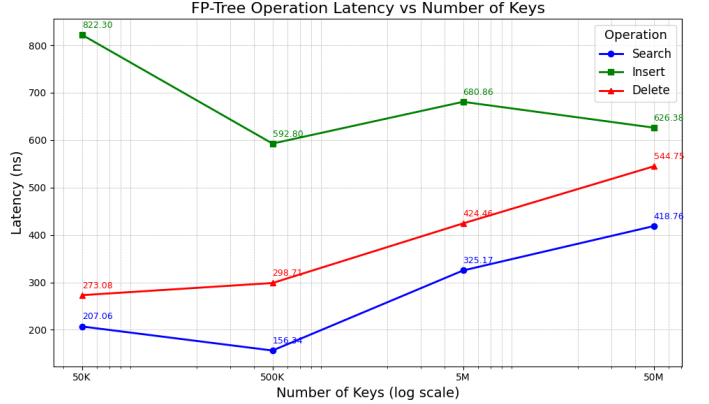


Fig. 3: Latencies obtained for FP-tree

For example in Figure 6, it is clear that for nDegree=800, for all the three operations of insert, search and delete there is a proper knee point. That is the latencies values are minimum.

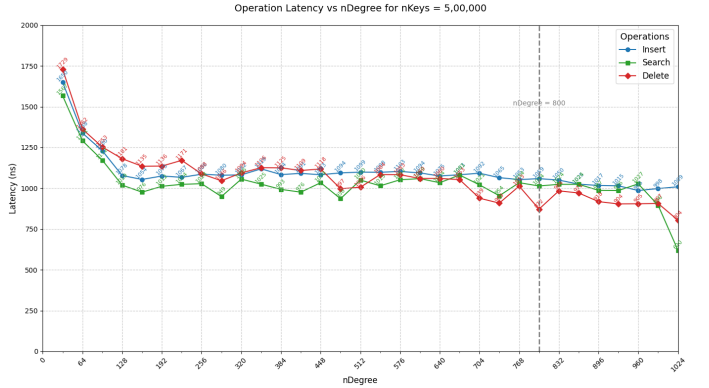


Fig. 4: Operation Latency vs nDegree for nKeys 5,00,000

Similar to this for all the nKeys a value of nDegree was found as the knee point according to the below table

nKeys	nDegree
50000	640
500000	800
5000000	320
50000000	704

TABLE IV: Best nDegrees

D. 4.4 Comparison of Latencies

In order to compare the performance of the FP-tree and the B+-tree on NVM, we compared their insert, search, and delete latencies. We compared them by determining the optimal nDegree parameters for the B+-tree for each key size nKeys by identifying the "knee point" for which the minimum latency is achieved. These nDegree parameters were used to compare B+-tree performance with the FP-tree for each operation.

1) **4.4.1 Insert Operation** : For the insert operation, we plotted the latencies of the B+-tree for four various $nDegree$ values (320, 640, 704, and 800) along with the FP-tree. Figure 5 illustrates the results.

Key observations include

- The FP-tree performs better than the B+-tree for frequent updates consistently. This is because of its selective write-back policy that minimizes writes to persistent memory and its light-weight log scheme that minimizes overhead.
- Among B+-tree configurations, larger values of $nDegree$ produce lower latencies due to the reduced tree height and less node splitting on insertions at the cost of increased memory usage.
- Both trees exhibit logarithmic growths of latencies with the number of keys, which speaks for their scalability to large data sets.

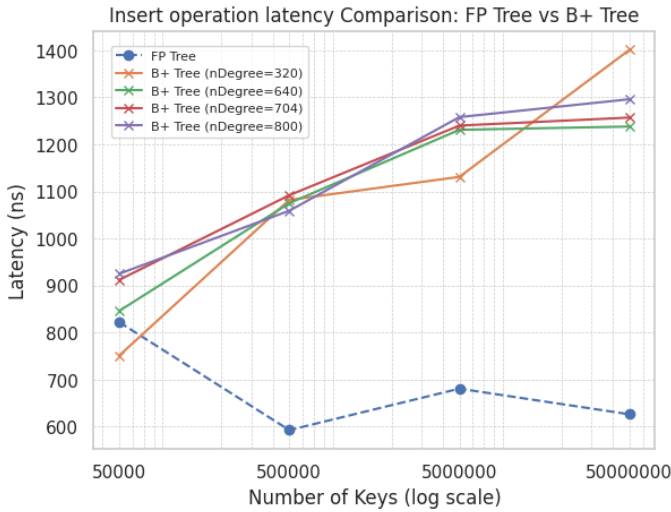


Fig. 5: Latencies for insert operations: Comparison of B+-tree (with varying $nDegree$) and FP-tree.

2) **4.4.2 Search Operation** : For the search operation, we repeated the same procedure, comparing the B+-tree's performance across the identified $nDegree$ values with that of the FP-tree. Figure 6 presents the results.

Key observations include:

- The B+-tree provides improved scalability for read-heavy workloads, particularly with higher $nDegree$ values. This is a result of the reduced tree height, which lowers cache misses and performs lookups more efficiently.
- The FP-tree performs competitively for small key sizes but has slightly higher latencies for large key sizes, perhaps because its hybrid SCM-DRAM architecture incurs overhead for certain access patterns.
- Both trees exhibit logarithmic growth in search latencies, attesting to their scalability for large-scale applications.

3) **4.4.3 Delete Operation** : Finally, for the delete operation, we compared the B+-tree's performance across the identified $nDegree$ values with that of the FP-tree. Figure 7 summarizes the findings.

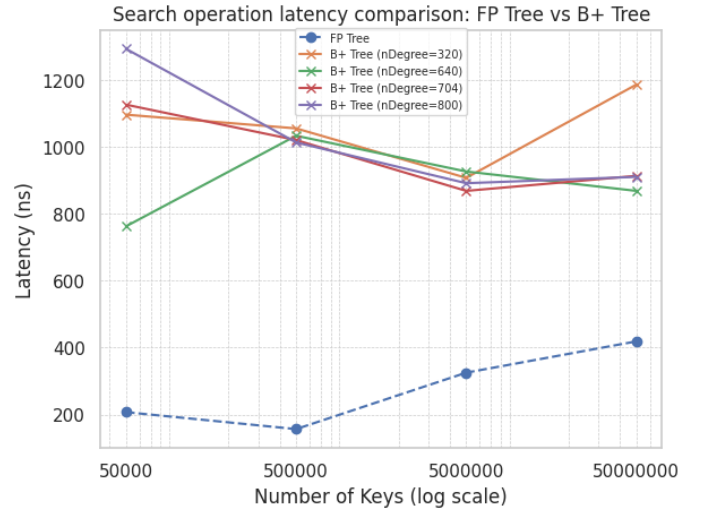


Fig. 6: Latencies for search operations: Comparison of B+-tree (with varying $nDegree$) and FP-tree.

Key observations include:

- As with the insert operation, the FP-tree outperforms the B+-tree for frequent updates. Its selective persistence and node merge efficiency are responsible for its lower latencies.
- For B+-tree configurations, higher $nDegree$ values again provide better performance, but the differences are less pronounced than for the insert and search operations.
- Both trees exhibit logarithmic growth in delete latencies, further confirming their scalability for large-scale applications.

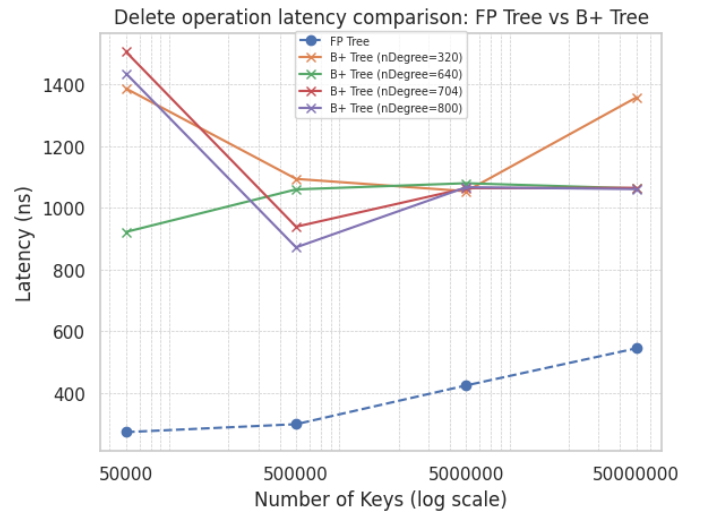


Fig. 7: Latencies for delete operations: Comparison of B+-tree (with varying $nDegree$) and FP-tree.

4) **4.4.5 Key Findings** : From the comparative analysis of insert, search, and delete operations, several key insights

emerge:

- **FP-tree Advantages:** FP-tree works better in the scenario of frequent updates since it employs a selective write-back policy and lightweight log mechanisms. These policies reduce write amplification and enhance durability, making it extremely suitable for hybrid SCM-DRAM systems.
- **B+-tree Scalability:** The B+-tree exhibits better scalability for read-heavy workloads, especially when configured with higher $nDegree$ values. Its ability to minimize tree height and optimize cache utilization makes it a strong candidate for large-scale data retrieval tasks.
- **Logarithmic Growth:** Both trees demonstrate logarithmic growth in latencies as the number of keys increases, making them suitable for large-scale applications. This is in keeping with their respective balanced tree data structures, delivering the guarantee of consistent performance even when dataset sizes grow.

E. 4.5 Profiling Results for B+-Tree

This section provides a brief outline on the results obtained from profiling of the B+-tree with different parameter configurations and key values. For profiling our tree binary, we switch back to our system with the specifications mentioned in the system setup. Once we identify knee points for each operation for each key value, we input this degree and key to the binary for hotspots and memory consumption analysis with VTune.

1) **4.5.1 Hotspots Analysis :** Hotspots analysis was performed on the binary using Intel VTune Profiler. For $nKeys=50000$, the suitable knee point obtained was for $nDegree=640$. Similarly, for $nKeys=500000$, the suitable knee point obtained was for $nDegree=800$. For $nKeys=5000000$, the suitable knee point obtained was for $nDegree=320$. For $nKeys=50000000$, the suitable knee point obtained was for $nDegree=704$. These data fed to the binary helped generate CPU Utilization graphs pertaining to each key value.

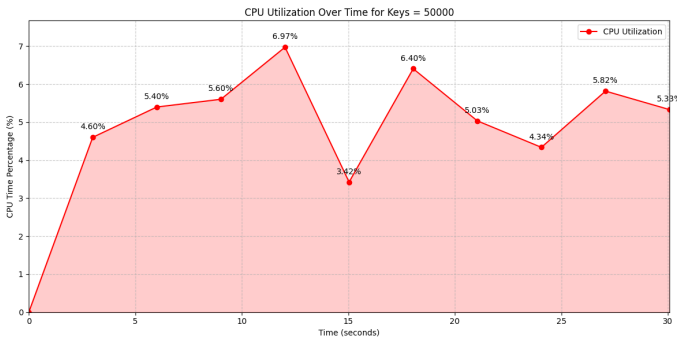


Fig. 8: CPU Utilization for keys=50000

2) **4.5.1 Memory Consumption Analysis :** Memory Consumption analysis was performed on the binary using Intel VTune Profiler. For $nKeys=50000$, the suitable knee point obtained was for $nDegree=640$. And, Similarly, for $nKeys=500000$, the suitable knee point obtained was for $nDegree=800$. These data fed to the binary helped generate

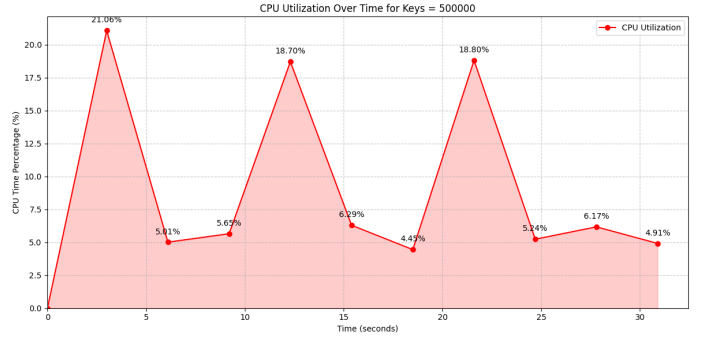


Fig. 9: CPU Utilization for keys=500000

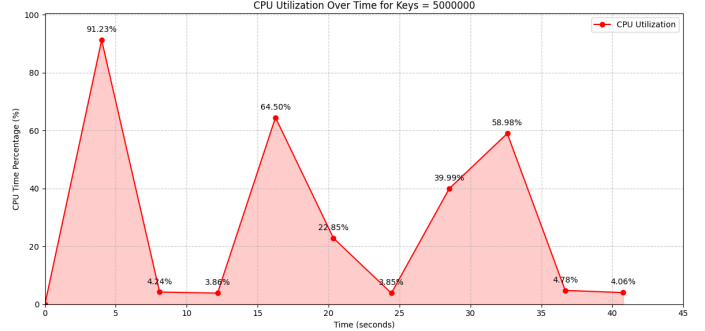


Fig. 10: CPU Utilization for keys=5000000

Memory Consumption graphs for tree operations pertaining to each key value.

From the memory consumption graphs, we infer that delete operations require more memory allocation out of the 3 operations. These graphs provide a detailed interpretation for memory consumptions of each function per key value.

F. 4.6 Single-thread micro-benchmarks on B+-tree with fixed-size strings

In this section, we focus on performance of single-threaded version of our B+-tree with base operations, but for fixed-size strings, i.e, fixed 16-byte string as keys and 8-byte integers as values. The same procedure is repeated to execute micro-benchmarks, as for the integer version, and the results obtained are used to create comparison graphs with the FPtree again for fixed-size strings. The FPtree was also micro-benchmarked for strings as well and the results plotted for comparison.

In order to compare the performance of the FP-tree and the B+-tree on NVM, we compared their insert, search, and delete latencies. We compared them by determining the optimal $nDegree$ parameters for the B+-tree for each key size $nKeys$ by identifying the "knee point" for which the minimum latency is achieved. These $nDegree$ parameters were used to compare B+-tree performance with the FP-tree for each operation.

Figure 14, 15, and 16 shows comparison of latencies for each operation of FPtree and B+-tree. On analyzing comparison results, we observed FPtree outperforms B+-trees for smaller key values, and for larger keys, performances are similar.

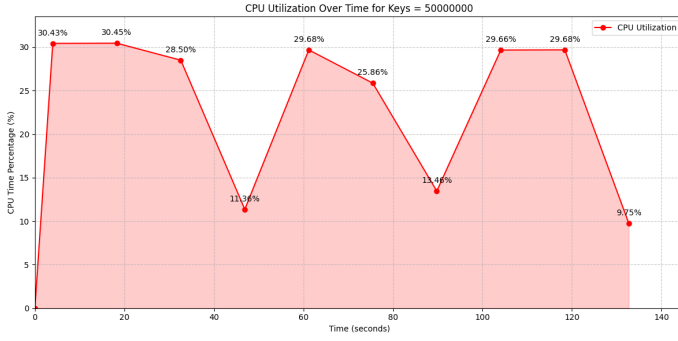


Fig. 11: CPU Utilization for keys=50000000

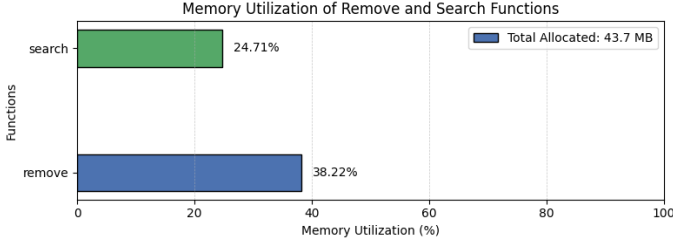


Fig. 12: Memory Consumption per function for keys=50000

V. CONCLUSION

In this study we evaluated the performance characteristics of an NVM-optimized B+-tree and compared it with the FP-tree, a hybrid B+-tree optimized for NVM-DRAM systems. Through a series of carefully designed benchmarks, we analyzed several key performance metrics, including operation latencies, CPU utilization, and memory consumption. The results made one thing clear: the FP-tree consistently outperformed our B+-tree across all tested operations, thanks to its hybrid memory-aware optimizations, selective write-back strategy, and efficient logging mechanisms. When handling insert, search, and delete operations, the FP-tree demonstrated significantly lower latencies, particularly at larger input sizes. This can be attributed to its ability to minimize writes to NVM and leverage DRAM as an effective caching mechanism. In contrast, the B+-tree struggled with higher write amplification and cache inefficiencies, making it less suited for hybrid memory environments in its current form. That said, the B+-tree still showed promise, particularly in its scalability when tuned for optimal configurations. Our profiling efforts helped pinpoint computational bottlenecks, shedding light on areas where performance could be improved. These findings not only highlight the strengths and weaknesses of different tree structures in hybrid memory settings but also pave the way for meaningful optimizations moving forward.

VI. FUTURE WORK

While this research provides valuable insights, there is still plenty of room for improvement and expansion. Based on our findings, we outline several key areas for future work:

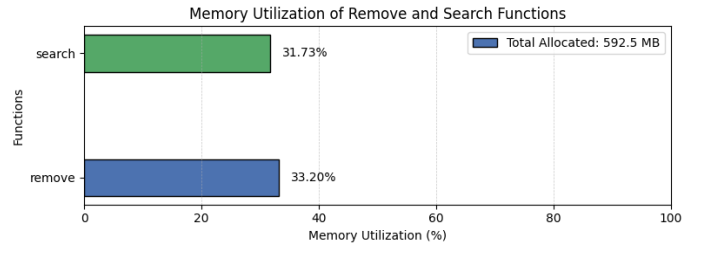


Fig. 13: Memory Consumption per function for keys=500000

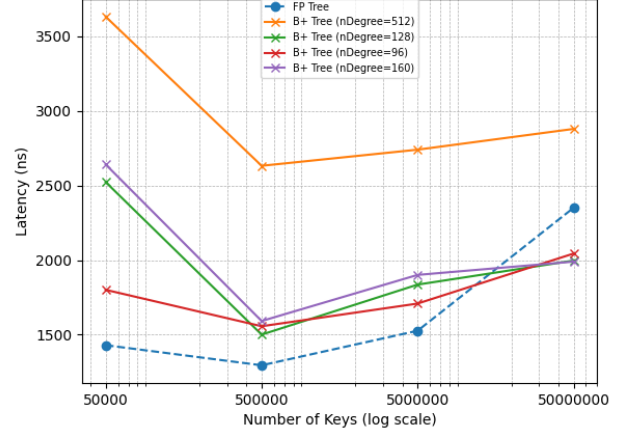


Fig. 14: Latencies for insert operations: Comparison of B+-tree (with varying $nDegree$) and FP-tree for fixed-size strings.

- **Implementing Zero-Copy Deserialization with the Cista Library** One of the biggest challenges we encountered was the overhead caused by serialization and deserialization. Right now, every time data is accessed, unnecessary copies are being made, leading to increased processing time and memory usage. To address this, we plan to integrate zero-copy deserialization using the Cista library, which would allow data to be accessed directly in memory without redundant copies. This change has the potential to significantly speed up operations and reduce memory overhead.
- **Supporting Variable-Length String Data** Our current implementation is limited to fixed-size keys and values. However, real-world databases often work with variable-length strings, such as names, addresses, and log data. Extending our B+-tree to efficiently handle variable-length string datasets will make it far more practical and applicable to real-world use cases.
- **Exploring Multi-Threading and Parallel Processing** Our current benchmarks were run in a single-threaded environment, but modern systems demand multi-threaded performance. Introducing parallelization and lock-free concurrency mechanisms could make the B+-tree significantly faster, especially for workloads with high transaction rates.

By implementing these improvements, we aim to push the performance of NVM-optimized B+-trees even further,

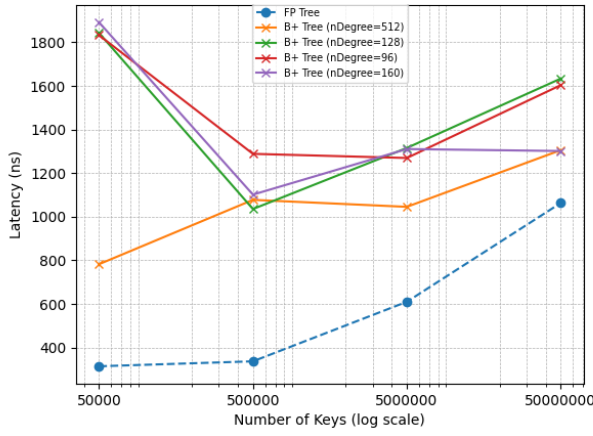


Fig. 15: Latencies for search operations: Comparison of B+-tree (with varying $nDegree$) and FP-tree fixed-size strings.

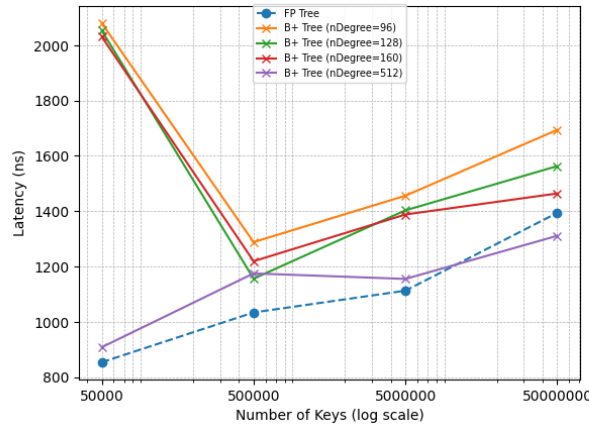


Fig. 16: Latencies for delete operations: Comparison of B+-tree (with varying $nDegree$) and FP-tree fixed-size strings.

making them more adaptable to hybrid memory environments. Ultimately, our goal is to bridge the performance gap between volatile and non-volatile storage, ensuring that these data structures can meet the demands of modern computing.

VII. ACKNOWLEDGEMENTS

We would also like to thank Sajad Karim (AG Datenbanken and Software Engineering, Otto-von Guericke University, Magdeburg) for his valuable suggestions and criticism throughout the project. We also acknowledge the contributions of Mustafa Abbas, Paul Blockhaus, Bala Gurumurthy, and David Broneske from the Faculty of Computer Science, Otto-von Guericke University, Magdeburg.

REFERENCES

- [BOUKHOBZA et al.(2017)] JALIL BOUKHOBZA, STÉPHANE RUBINI, RENHAI CHEN, and ZILI SHAO. 2017. Emerging NVM: A Survey on Architectural Integration and Research Challenges. (2017).
- [Chen(2016)] An Chen. 2016. A review of emerging non-volatile memory (NVM) technologies and applications. (2016).
- [Comer(1979)] Douglas Comer. 1979. Ubiquitous B-tree. ACM Computing Surveys. (1979).

- [Debbarma and Chandrasekaran(2016)] Tribid Debbarma and K Chandrasekaran. 2016. Comparison of FOSS based profiling tools in Linux operating system environment. (2016).
- [Friedman([n.d.])] Mark B. Friedman. [n.d.]. Measuring Processor Utilization in Windows and Windows applications. ([n.d.]).
- [Kakaraparth and Patel(2024)] Aarati Kakaraparth and Jignesh M. Patel. 2024. Fine-Grained Hardware Profiling – Are You Using the Right Tools? (2024).
- [Krótkiewicz and Mikłasz(2020)] Marek Krótkiewicz and Jakub Mikłasz. 2020. A Structure Based on B+ Trees to Represent a Large Number of k-Multisets Stored in Non-volatile Memory. (2020).
- [Lee et al.(2014)] Eunji Lee, Hyokyung Bahn, Seunghoon Yoo, and Sam H. Noh. 2014. Empirical Study of NVM Storage: An Operating System's Perspective and Implications. (2014).
- [Oukid et al.(2016)] Ismail Oukid, Johan Lasperas, Anisoara Nica, Thomas Willhalm, and Wolfgang Lehner. 2016. FPTree: A Hybrid SCM-DRAM Persistent and Concurrent B-Tree for Storage Class Memory. (2016).
- [R and B.(2016)] Elmasri R and Navathe S. B. 2016. Fundamentals of Database Systems. 7th ed. Pearson. (2016).
- [Spessot and Oh(2020)] Alessio Spessot and Hyungrock Oh. 2020. 1T-1C Dynamic Random Access Memory Status, Challenges, and Prospects. (2020).
- [Tsymbal and Kurylev(2017)] Vladimir Tsymbal and Alexandr Kurylev. 2017. Profiling Heterogeneous Computing Performance with VTune Profiler. (2017).
- [Yang et al.(2024)] Chen Yang, Jing Zhang, Xiguo Xie, and Jianpeng Sun. 2024. Research on Application Performance Analysis Methods for Scientific Computing. (2024).
- [Zaki and Hsiao([n.d.])] Mohammed J. Zaki and Ching-Jui Hsiao. [n.d.]. Software Support for Non-Volatile Memory (NVM) Programming. ([n.d.]). <https://doi.org/10.1137/1.9781611972726.27>